

University of Engineering and Technology, VNU, Hanoi
Institute for Artificial Intelligence



COURSE REPORT
INDUSTRIAL INTERNSHIP

**Topic: HumJob: A Local-First, Grid-Aware Pipeline for
Transcribing a Hummed Melody into Notes, Key and Chords**

Supervisor: PhD. Trần Hồng Việt

Author: Vũ Nguyên Đan - 23020351

Hà Nội, 2026

This image shows a full page of white paper with horizontal dotted lines, typical of primary school writing paper. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

Hà Nội, 2026

Supervisor

ACKNOWLEDGEMENTS

I would like to thank my supervisor for the guidance and feedback received throughout the Industrial Internship course, which shaped both the direction of this work, **HumJob**, and the standard of intellectual honesty I have tried to hold it to.

This report is the work of a single student. Given the breadth of the system and the limited time available, it inevitably contains shortcomings. In particular, the central finding of the evaluation is a negative one: the synthetic benchmark the project was tuned against no longer discriminates, while the system is not yet reliable on real hummed input. I have chosen to report that honestly rather than to present a saturated benchmark as a success. I welcome any comments and corrections from the instructor so that the work may be improved.

ABSTRACT

HumJob is a local-first system that transcribes a hummed melody into notation. It returns the note sequence, the estimated musical key, and a set of suggested chords, and exports them as MIDI, MusicXML, and engraved sheet music. The underlying problem is monophonic melody transcription from a bare human voice, which is hard because a continuous pitch contour has to be cut into discrete notes with correct pitches, onsets, and durations.

HumJob makes the problem tractable with two deliberate user-interface constraints. The user hums the syllable “da-da-da”, whose consonant gives every note a crisp onset and separates repeated notes of the same pitch; and the user hums to a metronome at a known tempo, which turns rhythm from estimation into snapping onto a known grid. The system is built as a chain of pure functions over three data types, with every tunable parameter centralised, and supports five interchangeable pitch backends (pyin, CREPE, PESTO, FCNF0++, and basic-pitch). A grid-aware segmenter and a backend-agnostic consolidation stage address the dominant failure mode, and a spectral octave-correction stage repairs a systematic pitch-tracker error on legato lines.

The system is evaluated on a synthetic fixture harness that scores note detection (pitch and onset F1) and, separately, rhythm (quantised onset and duration accuracy against an intended grid). On the expressive “realistic” fixtures the note F1 rose from 0.799 to 1.000 across three engineering iterations, and the harness is now saturated. This report treats that saturation as a limitation rather than a result: the synthetic benchmark no longer discriminates, informal testing on real hums still fails, and no labelled dataset of real hummed melodies exists to measure against. The rhythm evaluation isolates the concrete mechanism behind the real-world failures. Quantisation is perfect on-grid but degrades under human timing jitter (mean rhythm accuracy 0.937) and collapses under a 5% tempo error (0.602), and the tempo-detection component that is meant to prevent that error is a single global-tempo estimator that is fragile on real, variable-tempo humming. This report documents the architecture, the evaluation, and an honest account of what a saturated synthetic benchmark can and cannot demonstrate, together with a concrete plan for obtaining real ground truth.

Contents

1	Introduction	4
1.1	Motivation and significance	4
1.2	Objectives	4
1.3	Research questions	5
2	Background	5
2.1	Melody transcription and the monophonic subproblem	5
2.2	Fundamental-frequency estimation	6
2.3	From a pitch contour to notes	6
2.4	Key finding and rhythm quantisation	6
2.5	Why a metronome and “da-da-da”	7
3	System Design and Methodology	7
3.1	Architecture overview	7
3.2	The transcription pipeline	8
3.3	The note-detection backends	9
3.4	Grid-awareness: segmentation and consolidation	9
3.5	Spectral octave correction	10
3.6	The interactive editor (Manual mode)	10
3.7	Other independent paths	10
3.8	Optional AI coaching and the local-first exception	11
3.9	Tempo detection and its known fragility	11
3.10	Technology stack and running the system	12
4	Evaluation and Results	12
4.1	Evaluation methodology	12
4.2	The ground-truth problem	13
4.3	Note-detection results	13
4.4	The rhythm evaluation	14
4.5	Interpreting the scores	15
4.6	The tempo-detection weakness	15
5	Limitations and Future Work	16
5.1	Evaluation limitations	16

5.2	System deficiencies	16
5.3	Future directions	17
5.4	Towards production readiness	18
6	Conclusion	18
	References	20
	Appendix A: System demonstration	21
A.1	Transcriber: Auto mode	21
A.2	Transcriber: Manual mode	21
A.3	Pitch Finder	22
A.4	Realtime and vocal trainer	23
A.5	Transposer	23
A.6	Sing-Along and AI coaching	24

List of Figures

1	High-level system architecture of HumJob.	7
2	The HumJob transcription pipeline, both note-production paths.	8
3	Rhythm accuracy under timing jitter versus a wrong tempo.	15
4	Transcriber: Auto mode.	21
5	Transcriber: Manual mode.	22
6	Pitch Finder.	23
7	Realtime voice monitor and vocal trainer.	23
8	Transposer: hum mode.	24
9	Sing-Along and AI coaching.	24

List of Tables

1	The three data types the pipeline is built on.	8
2	The note-detection backends.	9
3	Technology stack.	12
4	Mean note F1 on the realistic fixtures across three iterations. The realistic take is the expressive one (wide vibrato, tremolo, drift, jittered consonant gaps, partial closures).	13
5	Per-fixture note metrics on the realistic take, current state. P, R, F1 are precision, recall, and F1 from mir_eval; ref/est is reference versus detected note counts.	14
6	Rhythm accuracy under two conditions. “on”, “dur”, and “both” are the fraction of notes with the correct onset, correct duration, and both; “err” is the mean onset error in milliseconds. The jitter condition applies human timing jitter (± 30 ms) at the correct tempo; the wrong-tempo condition applies a +5% tempo error to on-grid timing.	14
7	Known deficiencies of the current system and planned work.	17

1 Introduction

1.1 Motivation and significance

A common musical situation has no good tool. A person has a melody in their head, hums it, and wants it written down: as notes on a staff, as a MIDI file they can edit, or as a chord sketch they can play. Turning that hum into notation is the task known as *query-by-humming* transcription, and it is harder than it looks. The human voice produces a single continuous pitch contour that slides, wavers, and drifts; notation requires that contour to be cut into a sequence of discrete notes, each with a definite pitch, a definite start time, and a definite written duration, and it requires a key and a metrical grid within which those durations are meaningful.

Two families of existing tools sidestep parts of the problem rather than solving it. General audio-to-MIDI transcribers are trained on instruments and polyphonic music; on a bare voice they are prone to octave jumps and they do not exploit any of the structure a deliberate hum provides. Real-time pitch monitors and tuners report the instantaneous pitch but never commit to a note segmentation at all. What is missing is a system that is built specifically for a cooperative human who is willing to hum in a way that makes transcription tractable.

HumJob is that system. Its central design decision is to accept two small constraints on how the user hums, and to build the entire pipeline around them:

- The user hums the syllable “**da-da-da**”. The consonant closes the vocal tract briefly at the start of each note, which produces a crisp amplitude onset and, crucially, separates two repeated notes of the same pitch that a legato hum would smear into one.
- The user hums **to a metronome at a known tempo**. A known tempo grid converts rhythm from a hard estimation problem into a snapping problem: note onsets and durations are aligned to the nearest grid position rather than guessed from raw timing.

These constraints are load-bearing. They are what make the problem tractable for a single-student project, and a recurring theme of this report is that the remaining failures of the system are failures to exploit them fully, not failures of the underlying pitch models.

1.2 Objectives

The project pursues the following objectives:

- To design and implement a complete, local-first melody-transcription pipeline that takes a hummed recording and returns notes, key, and suggested chords, exported as MIDI, MusicXML, and engraved sheet music.
- To structure the pipeline as a chain of pure functions over a small, fixed data model, so that every stage is independently testable and every tunable parameter is centralised rather than scattered as magic numbers.

- To support several interchangeable pitch-detection backends behind a single interface, so the note-production model can be chosen or replaced without touching the rest of the pipeline.
- To deliver the pipeline both as a command-line tool and as a browser web application that records audio locally and calls the same code, with an interactive editor for correcting the automatic draft.
- To build an evaluation harness that measures the system honestly, and to characterise clearly what that harness can and cannot demonstrate.

1.3 Research questions

The project is organised around three research questions:

- **RQ1 - Do the interface constraints help?** Do the two deliberate constraints (a consonant onset per note, and humming to a known tempo grid) turn melody transcription from an estimation problem into a tractable segmentation and snapping problem?
- **RQ2 - Is this a segmentation problem or a model problem?** When notes come out wrong, does the fault lie chiefly in the segmentation and quantisation of an otherwise-correct pitch contour, rather than in the pitch model itself?
- **RQ3 - How well can the system be evaluated?** How well does the system agree with ground truth, and what are the methodological limits of that measurement given that the project has only synthetic ground truth and no labelled corpus of real hummed melodies?

2 Background

2.1 Melody transcription and the monophonic subproblem

Automatic music transcription in general is the conversion of an audio signal into a symbolic score. The general problem is polyphonic and very hard. HumJob addresses the restricted *monophonic* case: one voice, one note at a time. This restriction removes the need to separate simultaneous pitches, but it does not make the problem trivial, because the difficulty moves from source-separation to *segmentation*: deciding where one note ends and the next begins, and how long each note is in metrical terms.

A monophonic transcription pipeline has three logical stages: estimate a pitch contour over time, cut that contour into discrete note events, and place those events on a key and a rhythmic grid. HumJob follows exactly this shape, and the bulk of its engineering effort is in the second and third stages.

2.2 Fundamental-frequency estimation

The first stage estimates the fundamental frequency (f_0) of the voice, frame by frame. Several families of estimator exist and HumJob supports five of them behind one interface:

- **pYIN** [1] is a classical signal-processing estimator that augments the YIN algorithm with probabilistic thresholds and a Viterbi decoding step. It is fast, dependency-light, and the project default.
- **CREPE** [2] is a convolutional neural network trained to estimate pitch directly from the waveform, and is steady on singing voice.
- **PESTO** [3] is a self-supervised, transposition-equivariant pitch estimator; it is the web application’s default because it is precise on voice while being lighter and faster than CREPE.
- **FCNF0++** [4] is a fully-convolutional pitch and periodicity estimator, a precision peer of PESTO.
- **basic-pitch** [5] is an instrument-trained neural note-transcription model that emits note events directly, bypassing the frame-level contour.

The important background fact for this report concerns a specific failure of Viterbi-decoded trackers. On a continuously voiced legato line, with no silence to reset the decoder, the “stay put” transition prior of a Viterbi decoder can lock a whole note onto its sub-harmonic, reporting it a full octave too low. This is not an audio artifact; it is a decoding artifact, and Section 3.5 describes how HumJob detects and repairs it.

2.3 From a pitch contour to notes

Turning a contour into notes requires three sub-decisions. *Voicing* decides which frames carry pitch at all, gating out breaths and silence. *Segmentation* decides where the boundaries between notes fall, which for a “da-da-da” hum means finding the brief energy dips created by the consonant. *Consolidation* does the opposite where needed: a single held note that vibrato has fragmented into several pieces must be fused back into one. Segmentation and consolidation pull in opposite directions, and balancing them is the core difficulty of the pipeline.

2.4 Key finding and rhythm quantisation

Once notes exist, two musical decisions remain. *Key finding* identifies the scale the melody sits in; HumJob correlates the sung pitch-class distribution against the Krumhansl-Kessler probe-tone profiles for all twenty-four major and minor keys [6]. *Quantisation* snaps note onsets and durations to a metrical grid derived from the known tempo, which is what allows durations to be written as clean note values rather than as awkward fractions. When the tempo is wrong, quantisation produces non-integer durations that notation software renders as strings of tied notes, which is the visible symptom of most rhythm errors.

2.5 Why a metronome and “da-da-da”

The two constraints introduced in Section 1 are the project’s answer to the two hardest sub-problems above. The consonant in “da-da-da” directly supplies the onset cue that segmentation needs, and in particular it is the only reliable way to separate two consecutive notes of the same pitch. The metronome supplies the grid that quantisation snaps to. Neither constraint is burdensome for a cooperative user, and together they replace two estimation problems with two detection problems. The project treats them as fixed premises, not as limitations to be optimised away.

3 System Design and Methodology

3.1 Architecture overview

HumJob is organised as a chain of pure functions over three data types. Figure 1 shows the high-level structure: a core Python package implements the pipeline, a command-line tool calls it directly, and a FastAPI web application records audio in the browser and calls the same pipeline server-side. Everything runs locally, and no audio is ever sent to any third-party service. The single exception to local-only operation is an opt-in coaching feature (Section 3.8): only when the user explicitly presses a button does it send a short numeric summary of a sung practice take, never any audio, to an external language-model API.

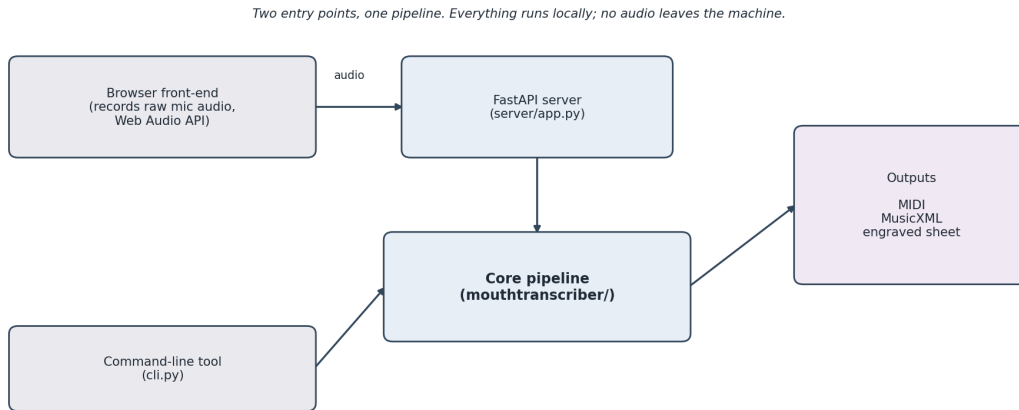


Figure 1: High-level system architecture of HumJob.

The three data types are the contract that every stage shares (Table 1). A **Frame** is one dense analysis frame, roughly one per 11.6 ms hop, carrying the frame time, the estimated f_0 , a voicing confidence, and a short-time energy. A **NoteEvent** is one discrete note after segmentation, carrying its start and end in seconds, its integer pitch, its raw fractional pitch, a cents offset from the nearest semitone, and, once quantised, its grid-aligned position and duration in quarter-note units. A **Score** is the whole transcription, ready to export.

Table 1: The three data types the pipeline is built on.

Type	Role
Frame	One analysis frame (≈ 11.6 ms): frame time, f_0 (NaN if unvoiced), voicing confidence, short-time energy. What the pitch tracker sees.
NoteEvent	One discrete note after segmentation: start/end seconds, integer MIDI pitch, raw fractional pitch, cents offset, and (after quantisation) grid position and duration in quarter-note units.
Score	The whole transcription: the note sequence, the detected key, tuning offset, and chords, ready to export.

A design rule the project holds to is that every tunable parameter lives in a single central parameter object, not as a magic number inside a stage. Changing behaviour means changing a named knob, which keeps the stages readable and the experiments reproducible.

3.2 The transcription pipeline

Figure 2 shows the pipeline stages in order. The chain is:

audio $\rightarrow$ preprocess $\rightarrow$ [note production] $\rightarrow$ consolidate $\rightarrow$ tuning $\rightarrow$ key $\rightarrow$ quantise $\rightarrow$ chords $\rightarrow$ export.

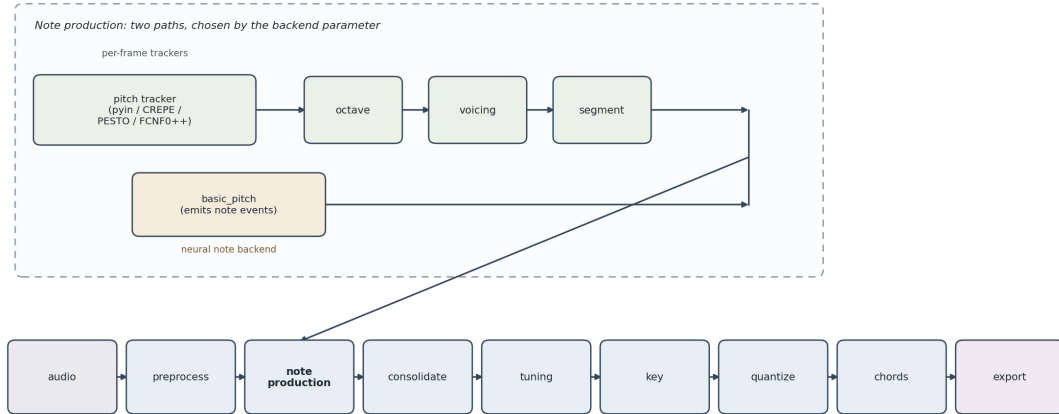


Figure 2: The HumJob transcription pipeline, both note-production paths.

Note production has two paths, selected by a backend parameter. The neural note-event backend (basic-pitch) emits notes directly and has no frame contour. The per-frame trackers (pyin, CREPE, PESTO, FCNF0++) produce a **Frame** contour that then passes through octave correction, a voicing gate, and the grid-aware segmenter to become notes.

Consolidation runs for every backend. It fuses the fragments that a held, vibrato-laden note leaves behind, which is the fix for the “one note becomes many slivers” failure. It is grid-aware: it will not fuse two same-pitch notes across a grid onset, so it never undoes a re-articulation that the segmenter deliberately split on the beat.

The remaining stages apply a global tuning correction, find the key by the Krumhansl-Kessler correlation, quantise onsets and durations onto the tempo grid, suggest chords, and export to MIDI, MusicXML, and engraved sheet music.

3.3 The note-detection backends

The five backends trade precision, speed, and training domain differently. Table 2 summarises them. The default differs by entry point, which is intentional: the library default is `pyin` (fast and dependency-light), the command-line default is `basic-pitch`, and the web application default is `PESTO` (most precise on voice). The four per-frame trackers all feed the same segmenter; only `basic-pitch` bypasses it.

Table 2: The note-detection backends.

Backend	What it is	Best for / notes
<code>pesto</code>	Self-supervised, transposition-equivariant pitch estimator	Most precise on voice; web-app default; lighter and faster than CREPE
<code>fcnf0</code>	FCNF0++, a fully-convolutional f_0 estimator	Precision peer of PESTO; shines on real voice; optional install
<code>crepe</code>	Neural pitch CNN, voice-trained	Steady on humming and singing; uses the accurate full model with Viterbi decoding
<code>basic_pitch</code>	Instrument-trained note-transcription CNN	Instrument clips; can octave-jump on a bare voice; bypasses the segmenter
<code>pyin</code>	Classical signal-processing f_0 plus the project segmenter	Crisp staccato “da-da-da”; library default

3.4 Grid-awareness: segmentation and consolidation

The single most important methodological idea in the pipeline is that segmentation is *grid-aware*. The segmenter is passed the known tempo and a fine short-window energy envelope. A short or soft consonant closure between two same-pitch notes appears as a narrow dip in that fine envelope; a width gate separates such a closure from a wide tremolo swell, and a shallower dip is still accepted when it lands on a beat, because the grid says a note boundary is expected there. This is what separates repeats of the same pitch, and it replaced an earlier coarse valley splitter that smeared over short closures.

Consolidation is the necessary counterweight. Because vibrato and breath can fragment one held note into several pieces, a backend-agnostic consolidation stage fuses same-pitch

fragments back together. Its grid-awareness is the safety interlock: it refuses to fuse across a grid onset, so segmentation and consolidation cannot fight each other into an oscillation.

3.5 Spectral octave correction

A distinct stage, inserted between the tracker and the voicing gate for the per-frame backends, repairs the Viterbi subharmonic error described in Section 2. The stage is backend-agnostic and rests on a spectral signature. A genuine fundamental at frequency f has energy at both its odd harmonics ($f, 3f, 5f$) and its even harmonics ($2f, 4f, 6f$). A subharmonic error, in which the tracker has reported half the true frequency, has energy only at the even multiples, because those are the true fundamental and its harmonics; the odd multiples of the reported frequency have little or no energy. The stage therefore computes, per voiced frame, the salience at the odd and even multiples of the reported f_0 , and doubles the reported frequency only when the odd-harmonic salience collapses relative to the even-harmonic salience. The test is deliberately conservative: it leaves genuine fundamentals alone, including missing-fundamental voices, which retain energy at their odd harmonics. Section 4.3 reports the effect of this stage on the evaluation.

3.6 The interactive editor (Manual mode)

The automatic draft is not always correct, and the weak link is segmentation rather than the pitch contour. The web application therefore provides a Manual mode: an in-browser, staff-based editor for fixing the automatic draft. It is almost entirely client-side. It re-implements the score-building and enharmonic-spelling logic of the export stage in the browser, engraves with a vendored WebAssembly build of the Verovio music-notation engine [9], and offers pure editing operations (change pitch, change duration, merge, split, delete to rest, insert) with undo and redo. A reference pitch strip shows the hummed contour against the chosen notes, so the user can see where the draft diverged from what they sang. The client sends the edited notes back to the server only at the edges, to re-export MIDI and MusicXML and to recompute key and chords.

3.7 Other independent paths

The web application is tabbed, and four tabs are separate paths that do not go through the transcription segmenter:

- **Pitch Finder** analyses an uploaded clip for key, tempo, and a Camelot code, using a chroma-based method that works on polyphonic audio. It reuses the key-scoring and tempo-folding code but not the monophonic segmenter, which would be wrong for a full song.
- **Realtime** is a client-side live pitch monitor and guitar tuner built on the Web Audio API, with a set of vocal-training drills (a target-note lane, a match game, a scale trainer, vibrato analysis, a vocal range finder, and a per-session progress log). A server round-trip cannot be real-time, so this path runs entirely in the browser and only asks the server for the sung key on stop, from a pitch histogram, with no audio upload.

- **Transposer** shifts a score to a new key. A file mode transposes an uploaded MIDI or MusicXML file server-side with music21 [8], moving every voice and the key signature; a hum mode transposes the last transcription client-side. The file mode also offers Camelot-compatible key presets as one-click shifts.
- **Sing-Along** turns an uploaded MIDI or MusicXML score into a karaoke exercise: the melody is played back as a guide while the user’s voice is tracked live and scored note by note. Because the whole melody is known in advance, the score itself drives the clock; there is no free-timing alignment. The server reduces a possibly polyphonic upload to a single monophonic melody line (its skyline, with ties stripped), and the browser schedules the count-in and the guide notes at absolute audio-clock times and scores the sung pitch against the active note. The in-tune tolerance is selectable (four difficulty levels), the guide volume is adjustable, and the take can be paused and resumed.

3.8 Optional AI coaching and the local-first exception

The Sing-Along tab includes an optional coaching feature, and it is the single place in the system where data leaves the machine. After a take, the client computes a deterministic, human-readable analysis of the performance entirely in the browser: the signed pitch bias (whether the singer was consistently sharp or flat), any drift from the start of the take to the end, octave slips, the accuracy of leaps versus stepwise motion, the weakest register, and the worst individual notes with their bar numbers. This analysis is always shown and requires no network access or key.

Only if the user then presses a “Get coaching” button is a compact, privacy-preserving numeric summary of that analysis, together with the musical context (key, tempo, time signature, chosen difficulty), sent to an external large-language-model API, which returns spoken-language feedback and practice suggestions in the user’s chosen language. The request contains numbers only: no audio, no recording, and not even the file name is transmitted. The API key is read server-side from a local, git-ignored configuration file and is never exposed to the browser, and the model’s reply is rendered as plain text rather than as markup, so untrusted model output cannot inject content into the page. The feature is off by default and does nothing until a key is configured, so the rest of the system remains strictly local; this one call is the deliberate, clearly-scoped exception to HumJob’s otherwise local-only operation.

3.9 Tempo detection and its known fragility

Because a wrong tempo is the usual cause of tied-note artifacts, the application offers a “find my tempo” button. Its purpose is narrow: the user hums a few even beats, the system estimates a comfortable tempo, and the user then records the real take to a click at that tempo, so that quantisation lands on whole beats.

The estimator computes an onset-strength envelope with librosa [7], then finds the dominant periodicity of that envelope by autocorrelation, weighted by a prior centred on a comfortable humming tempo, and folds the result into a plausible range by octaves. The important property, and a known weakness examined in Section 4.6, is that this returns

a *single global tempo* averaged over the whole clip, tilted toward the prior. It assumes a roughly constant tempo and reasonably clean onsets, and it is fragile when either assumption is violated.

3.10 Technology stack and running the system

Table 3 lists the main components. The whole system runs locally. The web application is a FastAPI server that serves a static browser front-end and reloads on source edits; the command-line tool transcribes a file directly.

Table 3: Technology stack.

Layer	Technology
Core pipeline	Python 3.12, NumPy
Pitch backends	pYIN via librosa [7]; torchcrepe [2]; pesto-pitch [3]; penn / FCNF0++ [4]; basic-pitch on ONNX Runtime [5]
Onset / tempo / chroma	librosa [7]
Score model, MIDI, MusicXML	music21 [8]
Engraving	Verovio [9] (server-side, and a vendored WebAssembly build in the browser)
Web backend	FastAPI, Uvicorn (with automatic reload)
Web front-end	Static HTML / CSS / JavaScript, Web Audio API
AI coaching (optional)	httpx client to an external, OpenAI-compatible language-model API (DeepSeek); off by default, optional, numeric summary only
Evaluation	mir_eval [10] for note metrics

4 Evaluation and Results

4.1 Evaluation methodology

The project’s standard evaluation is a synthetic fixture harness. It renders a set of known melodies as audio, runs the pipeline once per fixture, and compares the output against the known reference. The fixtures cover eight melodies: a C major scale, an A minor scale, an arpeggio, repeated notes, a melody with silence, octave leaps, a Twinkle excerpt, and a mixed-rhythm phrase.

The harness scores two different things, because they measure different failures:

- **Note detection (pitch and onset F1)**, computed with mir_eval [10]. A predicted note matches a reference note when its pitch is correct and its onset is within a tolerance; durations are ignored and no offset tolerance is applied. This is run in two flavours: a *clean* take that the regression gate expects to pass at $F1 = 1.0$, and a *realistic* take with wide vibrato, tremolo, pitch drift, jittered consonant-gap lengths, and partial consonant closures.

- **Rhythm**, added later, because note F1 is rhythm-blind. This scores the quantised onset and duration of each note against an intended grid and is described in Section 4.4.

4.2 The ground-truth problem

Before the results, their central limitation must be stated plainly. **All of the ground truth used in this report is synthetic.** The fixtures are melodies the project itself renders to audio, so their intended pitch, onset, and duration are known exactly by construction. This is a genuine strength for regression testing, but it is a genuine weakness for validity: a synthetic hum is more regular than a real one, and a benchmark built only on synthetic hums can be saturated while the system still fails on real input.

The project has no labelled corpus of real hummed melodies. The reason is specific to this task and is the same kind of problem that limits any transcription benchmark: obtaining ground truth for a real hum requires knowing the exact pitch, onset, and duration of every note the person intended, and a person cannot reliably label the pitch of their own hum by ear. As a result, every note-detection and rhythm figure in this report measures agreement with *synthetic* ground truth and must be read as a regression signal, not as a measure of real-world accuracy. Section 5.1 proposes a concrete remedy.

4.3 Note-detection results

On the clean fixtures the pipeline scores a mean note F1 of 1.000. The more informative number is the realistic take, where the segmenter historically struggled. Table 4 records the mean realistic F1 across three engineering iterations. The first raised it from 0.799 to 0.931 by making segmentation and consolidation grid-aware; the second raised it to 1.000 by adding the spectral octave correction of Section 3.5, which fixed the last failing fixture (the octave leaps, previously returned an octave flat).

Table 4: Mean note F1 on the realistic fixtures across three iterations. The realistic take is the expressive one (wide vibrato, tremolo, drift, jittered consonant gaps, partial closures).

Iteration	Change	Mean realistic F1
Baseline	coarse valley splitter	0.799
Grid-aware	grid-aware segment + consolidate	0.931
Octave correction	spectral subharmonic repair	1.000

Table 5 gives the per-fixture note metrics on the realistic take at the current state. Every fixture scores 1.000 on precision, recall, and F1, and the key is detected correctly in every case. This is the saturation referred to throughout this report: the note-detection half of the synthetic harness no longer discriminates between a better and a worse segmenter, so it can no longer drive improvement on its own.

Table 5: Per-fixture note metrics on the realistic take, current state. P, R, F1 are precision, recall, and F1 from mir_eval; ref/est is reference versus detected note counts.

Fixture	P	R	F1	Key	ref/est
c_major_scale	1.00	1.00	1.00	C major	8/8
a_minor_scale	1.00	1.00	1.00	A minor	8/8
arpeggio	1.00	1.00	1.00	C major	7/7
repeated_notes	1.00	1.00	1.00	C major	5/5
with_silence	1.00	1.00	1.00	C major	5/5
octave_leaps	1.00	1.00	1.00	C major	5/5
twinkle	1.00	1.00	1.00	C major	14/14
mixed_rhythm	1.00	1.00	1.00	C major	5/5
mean			1.000		

4.4 The rhythm evaluation

The note F1 above is rhythm-blind by construction: it ignores durations and accepts any onset within a tolerance, so it says nothing about whether the written rhythm is correct. This matters because the two failures reported most often on real hums are wrong rhythm (durations and onsets on the wrong beats) and over-splitting (one note returned as tied slivers), and neither is visible to note F1. A separate rhythm metric was therefore added. It compares the quantised onset and duration of each note against an *intended grid* read straight from each fixture’s specification, and reports the fraction of notes with the right onset, the right duration, and both. The headline number is “both”: right beat and right written length.

Two conditions were run, chosen to match the two things that actually break real hums. Table 6 gives the per-fixture results.

Table 6: Rhythm accuracy under two conditions. “on”, “dur”, and “both” are the fraction of notes with the correct onset, correct duration, and both; “err” is the mean onset error in milliseconds. The jitter condition applies human timing jitter (± 30 ms) at the correct tempo; the wrong-tempo condition applies a +5% tempo error to on-grid timing.

Fixture	Timing jitter (± 30 ms)				Wrong tempo (+5%)			
	on	dur	both	err	on	dur	both	err
c_major_scale	1.00	1.00	1.00	0	0.50	1.00	0.50	75
a_minor_scale	1.00	1.00	1.00	0	0.50	1.00	0.50	75
arpeggio	1.00	1.00	1.00	0	0.57	1.00	0.57	58
repeated_notes	1.00	1.00	1.00	0	1.00	0.67	0.67	0
with_silence	1.00	0.80	0.80	0	0.40	1.00	0.40	90
octave_leaps	1.00	1.00	1.00	0	1.00	1.00	1.00	0
twinkle	0.77	0.85	0.69	126	0.27	0.64	0.18	855
mixed_rhythm	1.00	1.00	1.00	0	1.00	1.00	1.00	0
mean			0.937				0.602	

Two findings stand out. First, on-grid at the correct tempo the quantiser is essentially perfect (a separate on-grid run, not shown, scores 1.000), so the quantisation logic itself is sound. Under human timing jitter the mean falls only to 0.937: most fixtures stay perfect, but the longest fixture, Twinkle, drops to 0.69 and loses a note, so drift bites longer pieces even at the right tempo. Second, and more importantly, a 5% tempo error collapses the mean to 0.602. Scales fall to 0.50, the silence fixture to 0.40, Twinkle craters to 0.18 with a mean onset error of 855 ms, and the repeated-notes fixture even loses notes (5 detected as 3), because a wrong tempo misfires the grid-aware segmenter as well as the quantiser.

Figure 3 plots the two conditions side by side.

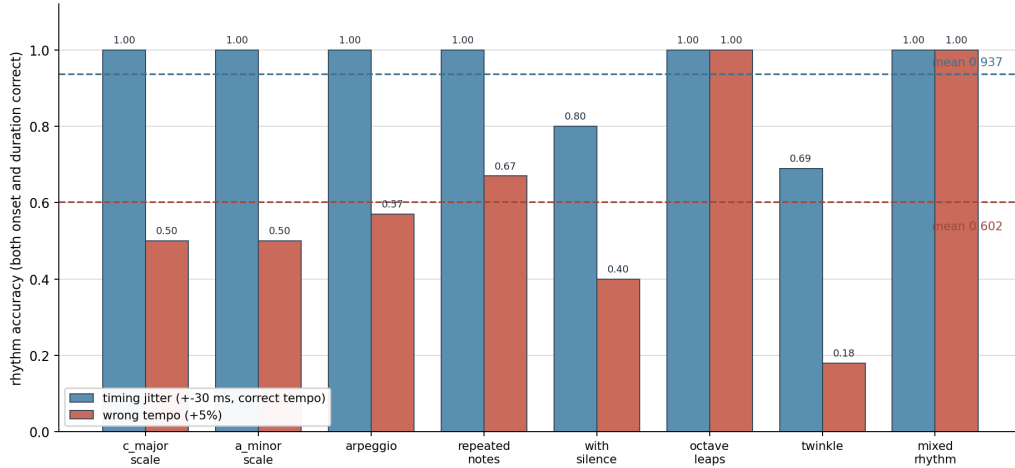


Figure 3: Rhythm accuracy under timing jitter versus a wrong tempo.

4.5 Interpreting the scores

The note-detection result and the rhythm result must be read together and read carefully. A saturated note F1 of 1.000 does not mean the system transcribes real hums correctly; it means the synthetic note-detection benchmark can no longer tell a good segmenter from a bad one. The honest reading is that the project has exhausted what its current synthetic note benchmark can teach, not that the transcription problem is solved. Informal use on real hums continues to fail, and there is no measurement in this report that contradicts that, because there is no real ground truth to measure against.

The rhythm evaluation is more useful precisely because it is not yet saturated. It localises the dominant real-world failure to a single cause: sensitivity to tempo error. A 5% tempo mismatch, which is well within the error of the system’s own tempo detector on real input, is enough to drive onsets off the integer beats, and onsets off the integer beats are what notation renders as tied slivers. This ties the user-visible “wrong rhythm” and “over-split” complaints to the same root cause and points directly at the tempo component as the highest-value target.

4.6 The tempo-detection weakness

The rhythm result above is a controlled injection of a tempo error. In real use the error is supplied by the system’s own tempo detector, and it is worth stating why that detector

is fragile, because it is the binding constraint on real-world rhythm accuracy.

As described in Section 3.9, the detector returns a single global tempo: it takes the autocorrelation of the onset-strength envelope over the whole clip, weights it by a prior centred near a comfortable humming tempo, and returns the dominant period. This has three consequences on real input. First, it assumes a constant tempo; a hum that speeds up or slows down has onset energy at more than one period, and the autocorrelation returns a blurry compromise rather than either true tempo. Second, the prior actively biases the estimate toward the centre of its range, so a genuinely slow hum is pulled upward. Third, it relies on clean, evenly spaced onsets, which real legato or noisy humming does not always provide. In informal testing, a hum of a roughly 76 beat-per-minute melody that included a faster passage was reported as about 101, a value suspiciously close to the prior, which is the signature of the prior dominating a weak autocorrelation. This weakness is the leading item of future work in Section 5.3.

5 Limitations and Future Work

5.1 Evaluation limitations

The main limitation is the one stated in Section 4.2: the evaluation is entirely synthetic, and no labelled corpus of real hummed melodies exists. Two concrete consequences follow. The note-detection benchmark is saturated and can no longer drive improvement. And the over-splitting failure, one of the two dominant real-world complaints, is still not measured at all, because synthetic vibrato and tremolo are too regular to produce the spurious narrow energy dips that real shimmer, breath, and creak produce.

The recommended remedy is not a larger synthetic set but a small set of real, labelled hums. The obstacle is ground truth, since a person cannot label the pitch of their own hum by ear. The plan, refined during this project, inverts the labelling. Rather than hum first and label afterward, the label is fixed before the recording: the user hums a melody they know, to a click at a known tempo, and the rhythm ground truth comes from the known tune and the click grid. The pitch of each note is confirmed with an external pitch-finder application used as an assistive aid, with the user verifying each note against the tune they knowingly hummed, rather than trusting the external tool as an oracle. This is important because an external pitch finder is itself another pitch detector; if it is trusted blindly and it happens to share an error with the pipeline, the evaluation would score that shared error as correct. Keeping the user in the loop, and drawing the rhythm truth from the click rather than from the tool, avoids that circularity. The resulting verified recordings, with their known notes and tempo, would give the project its first real ground truth.

5.2 System deficiencies

Several deficiencies of the current system are known and tracked. Table 7 lists them together with the planned remedy for each.

Table 7: Known deficiencies of the current system and planned work.

Area	Description and planned remedy
Tempo detection fragility	The “find my tempo” estimator returns a single global tempo biased by a prior, and is unreliable on real, variable-tempo, slow, or legato humming (Section 4.6). Planned remedy: flatten or remove the prior, and build the onset envelope from the pipeline’s own voicing onsets rather than raw spectral flux.
Over-splitting unmeasured	The over-split failure (one note returned as tied slivers or repeats) is not exposed by the synthetic harness because synthetic vibrato and tremolo are too regular. Planned remedy: add amplitude shimmer, breath, and creak to the synthesiser so spurious narrow dips appear.
No real ground truth	All evaluation is synthetic (Section 4.2); real-world accuracy is unmeasured. Planned remedy: the hum-to-a-known-score capture flow of Section 5.1.
Saturated note benchmark	The note-detection F1 is at 1.000 on all fixtures and no longer discriminates. Planned remedy: harder fixtures and, above all, real recordings.
Monophonic only	The transcription pipeline assumes a single voice, one note at a time. Polyphonic or chordal input is out of scope; the Pitch Finder tab uses a separate chroma-based method for full songs.
Optional backend availability	The most precise backends (PESTO, FCNF0++) are optional installs with heavier dependency trees, and FCNF0++ downloads weights on first use. The pipeline falls back to pyin when they are absent, at a precision cost.
Fixed metre assumption	Quantisation assumes a simple metre and a known, steady tempo. Time-signature changes and expressive rubato are not modelled.

5.3 Future directions

Three directions are recorded for the work after this report, in priority order.

- **(A) Tempo robustness (leading candidate).** The tempo detector is the binding constraint on real-world rhythm accuracy (Section 4.6), and it is directly testable because the fixtures have known tempos and the wrong-tempo rhythm pass is already the dashboard for it. Two cheap, high-impact changes: flatten or remove the tempo prior so slow tempos are not pulled upward, and build the onset envelope

from the pipeline’s own consonant onsets rather than from raw spectral flux.

- **(B) Real-hum ground truth.** The ultimate unlock, blocked only on the labelling method, which Section 5.1 resolves. This would let the project measure real-world accuracy for the first time and de-saturate the benchmark.
- **(C) Over-split synthesis.** Add amplitude shimmer, breath, and creak to the synthesiser so that the over-splitting failure becomes visible in the harness and can be regressed against.

5.4 Towards production readiness

HumJob is a research and academic project, not a production service. It runs locally, which is a deliberate design choice, but a wider deployment would additionally require packaging the optional neural backends so they install reliably, a broader automated test suite over real recordings, and error handling for the many ways a real microphone recording can be unusable (clipping, background noise, too quiet). None of these is conceptually hard, but together they are a meaningful body of work beyond the scope of this course project.

6 Conclusion

This project set out to transcribe a hummed melody into notes, key, and chords, by accepting two small constraints on how the user hums and building the whole pipeline around them. The result, HumJob, is a local-first pipeline of pure functions over three data types, with five interchangeable pitch backends, a grid-aware segmenter and consolidator, a spectral octave-correction stage, and a web application with an interactive editor, a pitch finder, a real-time vocal trainer, a transposer, and a sing-along trainer with optional, opt-in AI coaching.

With respect to the research questions, the honest verdicts are mixed. **RQ1** is supported by construction and by the evaluation: the two interface constraints do turn the hard parts into tractable ones, and the grid-aware stages that exploit them are what carried the realistic note F1 from 0.799 to 1.000. **RQ2** is supported: the diagnosed failures were segmentation and quantisation failures over an essentially correct pitch contour, not pitch-model failures. The clearest case is the octave error, which was proven to be a Viterbi decoding artifact rather than a fault in the audio, and was repaired by a spectral test rather than by changing the model. **RQ3** produced the most important lesson of the project, and it is a negative one. The synthetic benchmark the system was tuned against is now saturated at $F1 = 1.000$ while the system is still not reliable on real hums, because a synthetic hum is more regular than a real one and no labelled real corpus exists to measure against. The rhythm evaluation, which is not yet saturated, localises the dominant real-world failure to tempo sensitivity: the quantiser is perfect on-grid but collapses under a small tempo error, and the tempo detector that should prevent that error is a single global-tempo estimator that is fragile on real humming.

The main deliverable is therefore not a headline accuracy number. It is a working, well-structured pipeline together with an honest evaluation that says clearly where it stands: strong on a synthetic benchmark that has stopped being informative, and not yet validated

on real input. The most valuable next step is to build a small, trustworthy set of real, labelled hums, so that the system’s real-world accuracy can be measured directly rather than inferred from a saturated synthetic proxy.

References

- [1] M. Mauch and S. Dixon. *pYIN: A fundamental frequency estimator using probabilistic threshold distributions*. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2014.
- [2] J. W. Kim, J. Salamon, P. Li, and J. P. Bello. *CREPE: A Convolutional Representation for Pitch Estimation*. ICASSP, 2018. arXiv:1802.06182. Implementation: torchcrepe, <https://github.com/maxrmorrison/torchcrepe>.
- [3] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters. *PESTO: Pitch Estimation with Self-supervised Transposition-equivariant Objective*. International Society for Music Information Retrieval Conference (ISMIR), 2023. <https://github.com/SonyCSLParis/pesto>.
- [4] M. Morrison, C. Hsieh, N. Pruyne, and B. Pardo. *Cross-domain Neural Pitch and Periodicity Estimation*. 2023. arXiv:2301.12258. Implementation: penn, <https://github.com/interactiveaudiolab/penn>.
- [5] R. M. Bittner, J. J. Bosch, D. Rubinstein, G. Meseguer-Brocal, and S. Ewert. *A Lightweight Instrument-Agnostic Model for Polyphonic Note Transcription and Multipitch Estimation*. ICASSP, 2022. <https://github.com/spotify/basic-pitch>.
- [6] C. L. Krumhansl. *Cognitive Foundations of Musical Pitch*. Oxford University Press, 1990. (Krumhansl-Kessler / Krumhansl-Schmuckler probe-tone key profiles.)
- [7] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto. *librosa: Audio and Music Signal Analysis in Python*. Proceedings of the 14th Python in Science Conference (SciPy), 2015. <https://librosa.org/>.
- [8] M. S. Cuthbert and C. Ariza. *music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data*. ISMIR, 2010. <https://web.mit.edu/music21/>.
- [9] L. Pugin, R. Zitellini, and P. Roland. *Verovio: A library for Engraving MEI Music Notation into SVG*. ISMIR, 2014. <https://www.verovio.org/>.
- [10] C. Raffel, B. McFee, E. J. Humphrey, J. Salamon, O. Nieto, D. Liang, and D. P. W. Ellis. *mir_eval: A Transparent Implementation of Common MIR Metrics*. ISMIR, 2014. https://github.com/craffel/mir_eval.

Appendix A: System demonstration

This appendix shows the running web application from a user’s perspective. Each subsection corresponds to one tab. The screenshots are captured against a local instance of the stack; small visual details depend on the user’s browser and operating system.

A.1 Transcriber: Auto mode

The Transcriber tab records or uploads a hum and returns the automatic draft: the engraved sheet music, the detected key, and the suggested chords, with downloads for MIDI and MusicXML.

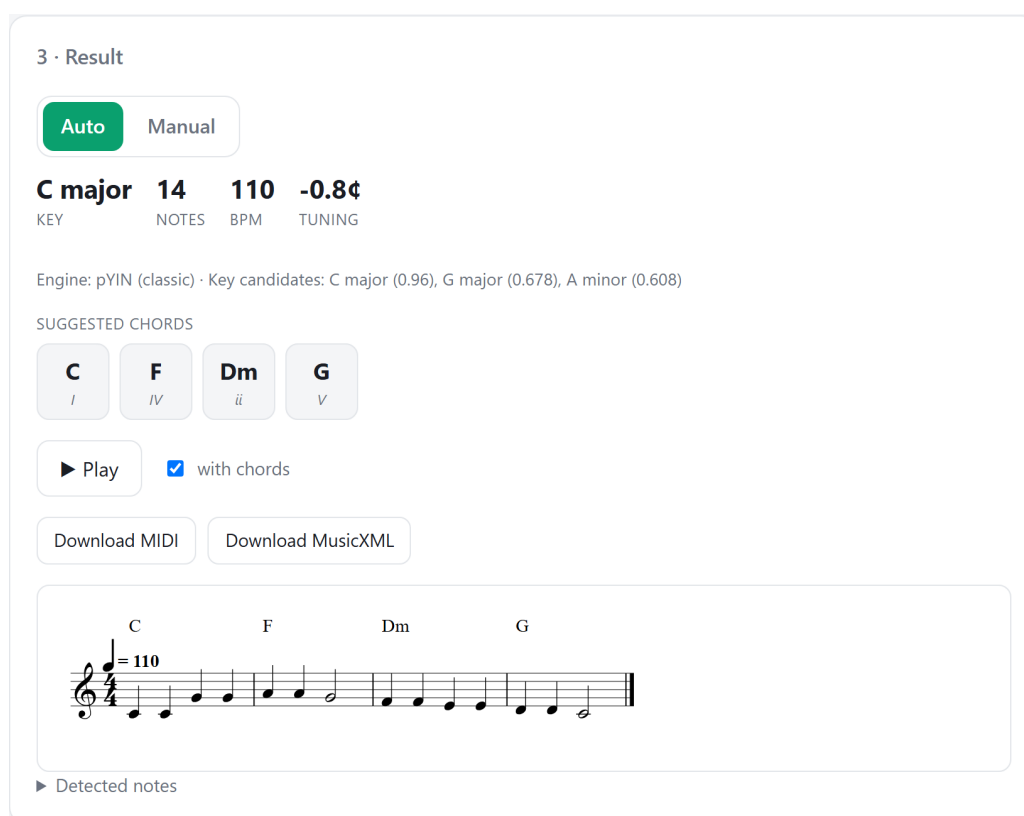


Figure 4: Transcriber: Auto mode.

A.2 Transcriber: Manual mode

Manual mode is the in-browser staff editor for correcting the draft. The reference pitch strip shows the hummed contour against the chosen notes, and the editing toolbar offers pitch, duration, merge, split, delete, and insert with undo and redo.

3 · Result

Auto Manual

Manual mode re-engraves the notes in your browser so you can fix them. Editing arrives next; for now it mirrors the Auto sheet.

C major 14 110 -0.8¢
 KEY NOTES BPM TUNING

Engine: pYIN (classic) · Key candidates: C major (0.96), G major (0.678), A minor (0.608)

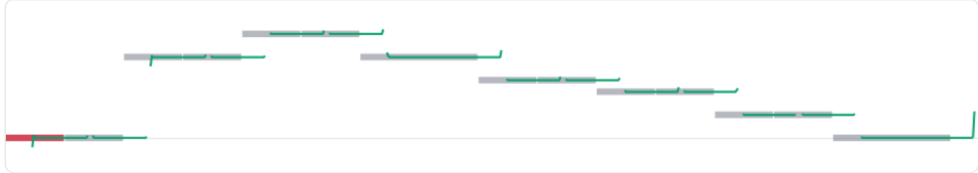
SUGGESTED CHORDS

C F Dm G
 I IV ii V

► Play ☒ with chords

Download MIDI Download MusicXML

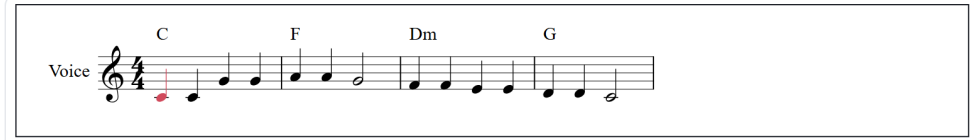
Reference: your hummed pitch (green line) vs the chosen notes (bars). Time runs left to right; this strip is its own time axis, not the staff's.



Note 1 of 14: C4

Pitch + semitone - semitone + octave - octave Length Longer Shorter Split Merge next

Delete Insert Undo Redo Revert to auto Update chords + key Download MIDI Download MusicXML



► Detected notes

Figure 5: Transcriber: Manual mode.

A.3 Pitch Finder

The Pitch Finder tab analyses an uploaded clip for key, tempo, and a Camelot code, using the chroma-based method that works on polyphonic audio.

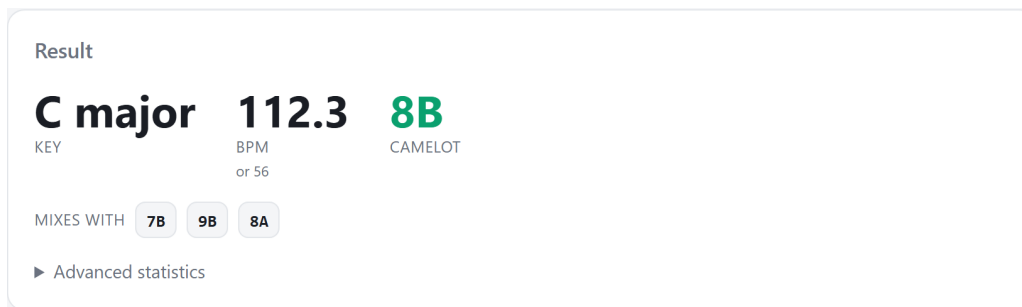


Figure 6: Pitch Finder.

A.4 Realtime and vocal trainer

The Realtime tab is a client-side pitch monitor and guitar tuner with vocal-training drills: a target-note lane, a match game, a scale trainer, vibrato analysis, a vocal range finder, and a per-session progress panel.

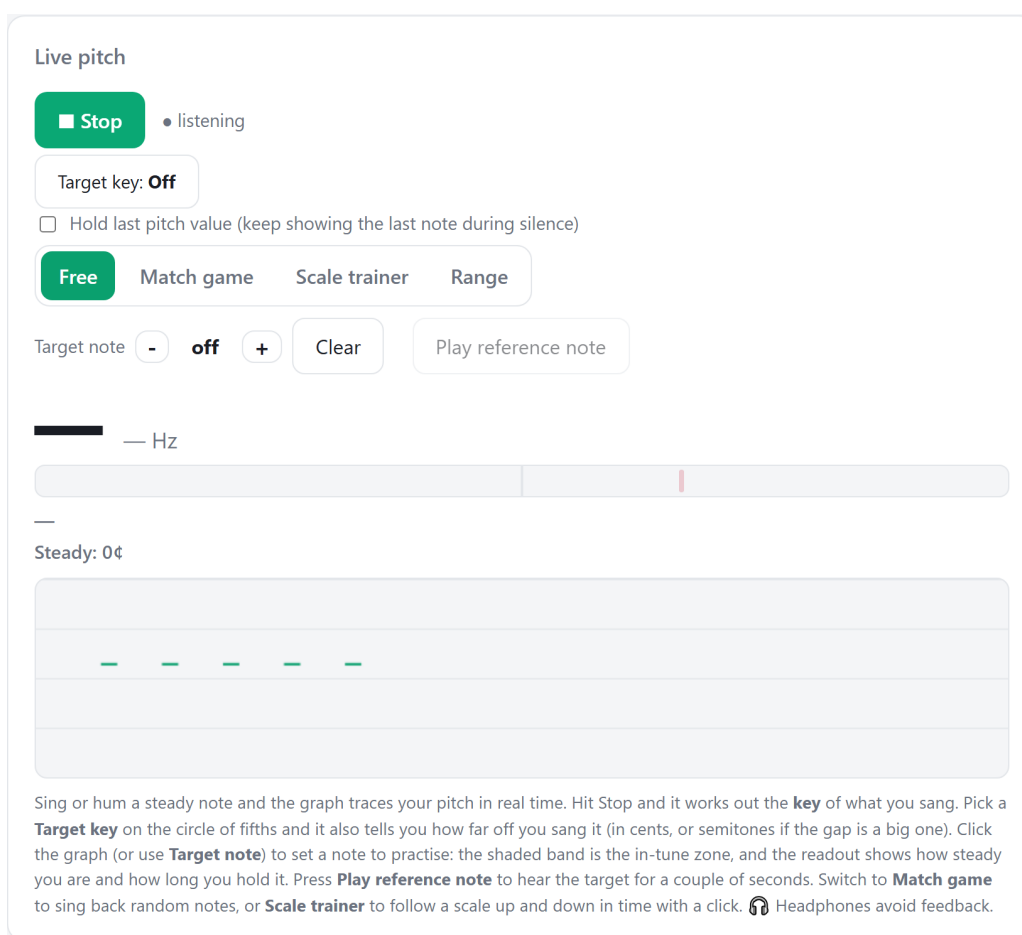


Figure 7: Realtime voice monitor and vocal trainer.

A.5 Transposer

The Transposer tab shifts a score to a new key. File mode transposes an uploaded MIDI or MusicXML file server-side and offers Camelot-compatible key presets; hum mode trans-

poses the last transcription client-side.

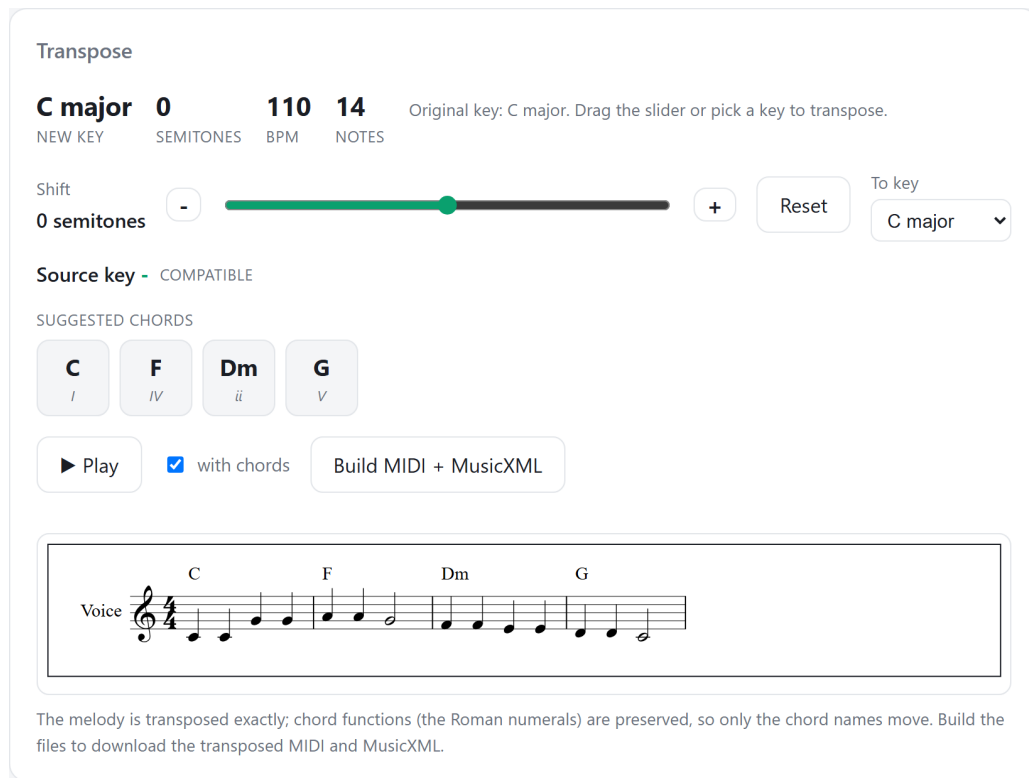


Figure 8: Transposer: hum mode.

A.6 Sing-Along and AI coaching

The Sing-Along tab plays an uploaded score as a guide, tracks the voice live, and scores each note. After a take it shows a deterministic analysis of the performance offline, and, on request, sends a numeric summary to an external language model for spoken-language coaching (the one non-local, opt-in feature).

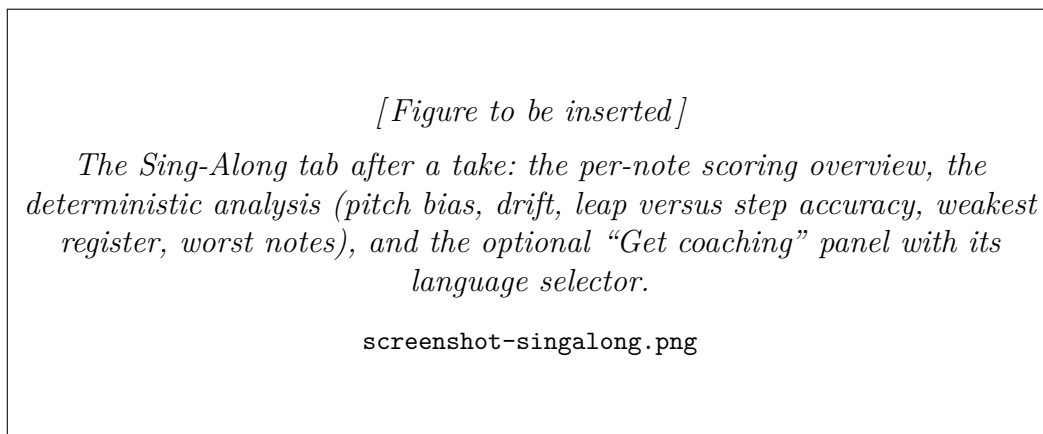


Figure 9: Sing-Along and AI coaching.